

Open source as resume

Week 8, Lecture 1 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- How to find a project worth contributing to
- What to read before you write a single line of code
- The anatomy of a first PR that gets merged
- How to become a regular contributor after the first merge
- How to cite open source work in an interview

By the end of this lecture you can find a real project, open a PR with a description that helps a maintainer say yes, and explain the contribution in an interview.

Finding the right project

Not all projects are equal for your purposes

- The goal is a contribution that a target company will recognize as relevant
- A project the company maintains, depends on, or respects is worth ten times more than a random popular repo
- Check: does the company's engineering blog mention this project? Does it appear in their public job descriptions?
- Relevance to your target list is the first filter, before anything else

Start with your target-company list and work backwards to projects. (GitHub Open Source Guides, 2023)

Reading the health signals before you commit

Four signals that predict whether a contribution will go anywhere:

1. Recent commit activity: at least one commit in the past 30 days
2. Open PR response time: maintainers leave comments within a week or two
3. Issue tracker quality: issues are labeled, triaged, and closed when fixed
4. A CONTRIBUTING.md or CONTRIBUTORS guide that is actually maintained

A stale repository with 10,000 stars is not a healthy project. A 200-star project where maintainers respond in 48 hours is.

Finding your first issue

- Search for labels: `good first issue`, `help wanted`, `beginner friendly`, `docs`
- Read the issue thread before claiming the issue; someone may already be working on it
- If no labels exist, look for small, scoped documentation gaps or test coverage gaps
- "A common misconception about contributing to open source is that you need to contribute code. In fact, it's often the other parts of a project that are most neglected." (GitHub Open Source Guides, 2023)

Non-code contributions get merged faster and teach you the project's workflow before you touch production code.

Reading before writing

CONTRIBUTING.md is the contribution contract

- Every project with active maintainers has a contribution guide, often at CONTRIBUTING.md in the root
- Read it before forking. Read it again before opening a PR
- Common requirements you will miss if you skip it: commit message format, branch naming convention, test requirements, DCO sign-off, linked issue requirement

Skipping the contribution guide is the single most common reason first PRs are closed without review.

What a contribution guide tells you

Five things to extract from any contribution guide:

1. How to set up the local development environment
2. Whether you need to link a PR to an issue first (many projects require this)
3. The expected commit message format (often Conventional Commits or a project-specific style)
4. Which tests you must run and how to run them locally
5. Who reviews PRs and how long reviews typically take

Write these down before you start. Knowing the rules in advance means your PR arrives complete, not in need of three rounds of fixes.

The first merged PR

The fork-branch-commit-PR workflow

1. Fork the repo to your own account
2. Clone your fork locally
3. Create a branch named for the issue: `fix/123-null-pointer` or `docs/update-readme`
4. Make one atomic change
5. Run the full test suite locally
6. Push the branch to your fork
7. Open a PR from your fork's branch to the upstream main

One change per PR. Do not bundle two fixes. Maintainers merge focused PRs faster. (first-contributions, Jossey et al., 2017)

The PR description that earns a merge

A PR description does three things:

1. **Names the problem.** "Fixes #123: the login handler throws a null pointer when the session token is missing."
2. **Describes the change.** "Added a null check in `auth/session.go` line 47 before dereferencing the token. Returns a 401 with a descriptive message instead of a 500."
3. **Shows evidence.** "Ran `go test ./...` locally, all tests pass. Added one unit test covering the null token case."

Optional but useful: flag any open questions you have for the maintainer. This shows judgment, not uncertainty.

What maintainers respond to

Things that get PRs merged faster:

- A description that explains the *why*, not just the *what*
- A single, focused change with no unrelated edits
- Tests that cover the change
- A response to review comments within 24-48 hours
- Patience: reviews take time, and following up after two weeks is fine

Things that get PRs closed:

- No linked issue
- A change that violates the contribution guide
- A PR that touches ten files for a one-line fix
- No response to reviewer questions after two weeks

Becoming a regular contributor

The path from one PR to trusted contributor

One merged PR is a line on a resume. A pattern of contributions is a relationship.

Steps after the first merge:

1. Leave a brief thank-you comment; ask if there is a higher-priority issue you could help with next
2. Subscribe to the issue tracker; triage new issues if the project allows it
3. Review other contributors' PRs with substantive comments, not just approvals
4. Take on a slightly larger issue for PR 2: a small feature or a refactor the maintainers have flagged

Regular contributors get reviewed faster. Maintainers remember names. That memory is a referral surface. (GitHub Open Source Guides, 2023)

What sustained contribution looks like in practice

Realistic cadence for a student contributor:

- One PR per month is enough to stay visible
- Two to three issues reviewed per week takes 20 minutes
- A comment thread on a design issue shows technical judgment with no code required
- Documentation improvements are always welcome and always lag

The bar for "regular contributor" is lower than most students think. Showing up consistently at a low volume beats a burst of activity followed by silence.

Citing OSS work in interviews

How to frame a contribution in an interview

Three elements every interviewer needs to understand:

1. **What the project does and who uses it.** One sentence. Show that you understood the context before contributing.
2. **What problem you solved.** Be specific about the bug, gap, or improvement. Name the function, the file, or the user-facing symptom.
3. **What the outcome was.** "The PR was merged in week 2 of my contribution. The fix is now in version 3.4 and shipped to all users."

If the PR was not merged, say so honestly and explain what you learned from the review comments. An unmerged PR with good reviewer feedback still shows you can engage with production-quality code.

The interview question this answers

"Tell me about a time you contributed to something larger than yourself."

Open source is the cleanest answer to this question a new grad can give:

- The code is public and verifiable
- The review comments show how you respond to feedback
- The project context shows you can read an unfamiliar codebase
- The contribution history shows sustained effort, not a one-time event

One merged PR with a clear description and a good story is worth more than ten closed PRs you mention vaguely.

Take-aways

1. Find a project your target companies will recognize. Relevance and maintainer health are the two filters.
2. Read CONTRIBUTING.md before touching code. Skipping it is the most common reason first PRs are closed.
3. One atomic change per PR. A focused PR with a clear description and tests gets merged; a sprawling one gets feedback requests that drag on for weeks.
4. Sustained contribution at a low volume beats a burst of activity. One PR a month over three months is a relationship.
5. An interview story about a merged PR is one of the strongest portfolio signals a new grad can produce.

"A common misconception about contributing to open source is that you need to contribute code. In fact, it's often the other parts of a project that are most neglected."

GitHub Open Source Guides, *How to Contribute to Open Source* (2023)

What's next

- **Section (this week):** Triage and ship. Pick one move (OSS PR draft, three cold emails, or one public artifact) and bring what you shipped to peer review.
- **Reading:** Week 8 reading covers open source contribution, unpaid stints, find-a-problem, and build-in-public at </c/landing-the-offer-26au/readings/wk08>
- **Lecture 2 (this week):** Unpaid stints and find-a-problem moves
- **HW 4 (system design write-up):** due this week